1. Strategy Pattern

We need to implement using the Strategy Pattern a system that provides the basic operations with integers (+, -, /, *). The two integer values are read and afterwards the user enters the given operation and the result is displayed.

Solution: https://github.com/loose-good/design-patterns/tree/main/src/main/java/p1

2. Template Method Pattern

In the code given below we can find a sequence of duplicated code between the two methods belonging to the implemented two classes. Unfortunately the two methods (makeStuff) are not identical and have minor differences. Refactor the code in order to remove the duplicated code - you are allowed to modify the given classes (e.g. add new attributes and new methods) but you are not allowed to add new classes.

```java
class Worker {
   protected int x;

   public Worker(int _x) {
       x = _x;
   }

   public int makeStuff() {
       int z;
       x++;
       z = x * x;
       System.out.println("Values are:" + x + " and " + z);
       return x + z;
   }
}

class SpecializedWorker extends Worker {
   private int y;

   public SpecializedWorker(int _x, int _y) {
       super(_x);
       y = _y;
   }

   public int makeStuff() {
       int z;
       x += y;
       z = x * x + y * y;
       System.out.println("Values are:" + x + " and " + z);
       return x + z;
   }
}
```

Solution: https://github.com/loose-good/design-patterns/tree/main/src/main/java/p2

3. Observer Pattern

In Jira, a well known issue tracking manager, we can create, close issues and add comments to issues. The state of a new issue is unsolved and the state of a closed issue is closed. Managers like numbers and they want to see every time an issue is created/closed the total number of the unsolved and closed issues, as it follows:

- they want to be informed via email about the values of these two numbers every time an issue is created/closed
- they want to see the values of these numbers inside a bar chart that is always displayed on their screen

Which design pattern do you think is useful in order to help managers? Present the main participants for the general case and present the actions that have to be performed if we want to add an additional visualization (a table) managers can look at? Draw a uml diagram.

Solution: https://github.com/loose-good/design-patterns/tree/main/src/main/java/p3

4. Command Pattern

We have a remote controller that has three entries (positions) and each entry has two buttons: on and off. We need to configure each button with some actions, as it follows: Button 1 on/off - turn the TV on/off Button 2 on/off - turn the media player on/off Button 3 on/off - turn the media player and TV on/off The class that models the TV has two methods providing the needed services named turnTVOn() and turnTVOff(). The class that models the media player has two methods providing the needed services named turnOn() and turnOff(). Name a design pattern that helps in order to build the configurable remote controller and present the involved participants for the general case. For the aforementioned example draw the uml diagram.

Solution: https://github.com/loose-good/design-patterns/tree/main/src/main/java/p4

5. Visitor Pattern

Imagine you are developing a shape drawing software where you currently support drawing Circles and Triangles. The internal structure of the two classes looks as follows:

```
@Data
class Point {
   private int x, y;
}

@Data
class Circle {
   private Point center;
   private double radius;
}

@Data
class Triangle {
```

```
    private Point point1, point2, point3;
}
```

Your tool is wonderful but now management asks you to export your drawings to a file, so other tools are able to interpret and analyze your drawings. The problem is, you don't know what format the other tools require in order to be able to read the exported file. You are given the job to create a flexible design of your software so that it will be easy to accommodate new export formats, as well as new Shape types.

The first two clients of your application require the following two formats:

1. XML:
    a. Triangle:
    ```
    <triangle>
        <points>
            <point>
                <x>3</x>
                <y>6</y>
            </point>
            <point>
                <x>2</x>
                <y>7</y>
            </point>
            <point>
                <x>-1</x>
                <y>10</y>
            </point>
        </points>
    </triangle>
    ```
    b. Circle:
    ```
    <circle>
        <radius>10</radius>
        <center>
            <point>
                <x>0</x>
                <y>0</y>
            </point>
        </center>
    </circle>
    ```
2. JSON:
    a. Triangle:
    ```
    {
        "type": "TRIANGLE",
        "points": [
            {"x": 3, "y": 6},
            {"x": 2, "y": 7},
            {"x": -1, "y": 10}
    ```

```
        ]
    }
```

b. Circle:

```
{
    "type": "CIRCLE",
    "radius": 10,
    "center": {"x": 0, "y": 0}
}
```

Draw the UML diagram, together with important code snippets. Write the code that supports this scenario. After that, create a Main class where you create a Triangle and 2 Circles and export them to both JSON and XML formats.

Add a new shape called Rectangle that has the following internal structure:

```
@Data
class Rectangle {
   private double height, width;
}
```

What other classes should you have to add? Export a Rectangle to JSON and XML.

```
<rectangle>
   <width>20</width>
   <height>53</height>
</rectangle>
```

```
{
   "type": "RECTANGLE",
   "width": 20,
   "height": 53
}
```

Add a new export format called YML. What has to change? YML format has the following structure:

```
elements:
 - circle:
     radius: 10
     center:
       x: 0
       y: 0
 - rectangle:
     height: 20
     width: 53
```

```yaml
- triangle:
    points:
      - point:
          x: 3
          y: 6
      - point:
          x: 2
          y: 7
      - point:
          x: -1
          y: 10
```

Solution: https://github.com/loose-good/design-patterns/tree/main/src/main/java/p5

6. Decorator Pattern

Implement a system that allows us to model different types of pizzas. Details about the components of a pizza should be printed and each pizza has a cost. The basic pizza that is served is SimplePizza. The components of this pizza are ketchup and mozzarella and the cost of a simple pizza is 10 RON. We have different ingredients that can be added to a pizza and we can add any existing ingredient (or any possible combination) to a SimplePizza. For the sake of simplicity current we have only the next ingredients:

- Ham - 5 RON
- Extra cheese - 5 RON
- Mushroom - 4 RON
- Salami - 5 RON

Solution: https://github.com/loose-good/design-patterns/tree/main/src/main/java/p6

7. Composite Pattern

Implement a system that allows us to create arithmetic expressions like:

- 3
- (3 + 7)
- (3 * 7)
- (3 * (7 - 5))
- ((3 + (7 + 5))*3)

Implement the hierarchy of expressions that provides a service for printing an expression as well as a service for computing the value of an expression.

Solution: https://github.com/loose-good/design-patterns/tree/main/src/main/java/p7

8. Abstract Factory Pattern

We need to model a robot responsible for assembling a car. For the sake of simplicity, we consider that a car has only three components: a Car Body, an Engine and a Brake.

- An Engine can be a *TSI 1.2 Engine*, a *TSI 1.8 Engine* or a *Diesel Engine*.
- A Brake can be a *normal brake* or a brake supporting the *Hill Hold Control Feature.*

- The Car Body can be a *Simple Car Body*, a *Sport Body* or a *Combi Body.*

The Robot provides a single functionality: assembleCar(). For example, in the next fragment it is provided a possible implementation for assembling a Combi TSI 1.2 Car with a normal Brake:

```
class Robot {
   public void assembleCar() {
       Engine e = new TSI12Engine();
       Brake b = new NormalBrake();
       CarBody c = new CombiBody();

       System.out.println("Assemble the car");
   }
}
```

What is wrong with the provided implementation? Modify the implementation using the Abstract Factory Pattern. Give the implementations for your factories for assembling

- a Combi TSI 1.2 Car with a normal Brake
- a Sport TSI 1.8 Car with a Brake supporting the Hill Hold Control Feature.

Solution: https://github.com/loose-good/design-patterns/tree/main/src/main/java/p8

9. Factory Pattern

Inside a game we can build different mazes. Each maze has a width and a height and inside each maze we have width * height cells. Each cell has entries and walls. The maximum number of walls and their positions are given when the maze is instantiated and randomly computed when the maze is instantiated. We may have mazes with red, blue or green walls (all the walls always have the same colour) and the user sets the colour when the game starts. Each wall is able to draw itself as a rectangle, the width and height of the rectangle being 10 and 20 if the class is instantiated using the default constructor. The wall has services for changing its attributes. Inside the maze we have a person who can be represented as a soldier or a princess, the type of the person being a setting of the game. Draw the class diagram and present different creational patterns that can be used in order to facilitate the construction of a maze.

Solution: https://github.com/loose-good/design-patterns/tree/main/src/main/java/p9/diagram